

The inventory nobody has

Source of truth

Multi-vendor

17 September 2026 · 8 minute read

Almost every conversation we have with a network team starts the same way: they do not know for certain what is on their network. There is a spreadsheet, and nobody trusts it. This article is about why that happens, the difference between an inventory and a source of truth, and what becomes possible once the network finally describes itself.

The spreadsheet nobody trusts

The pattern repeats with a consistency that is hard to ignore. There is a file with the list of devices. It was built by someone who has since left, or built for an audit three years ago. It has columns for model, operating system version, site, and management address. And it invariably has three problems: devices that no longer exist, devices that exist and are not listed, and data that used to be true.

The interesting part is not that the file is out of date. It is that *everyone knows* it is out of date, and it is still used to make decisions. A maintenance window is planned against that list. An audit is answered with that list. A technology refresh is sized with that list.

The reason nobody fixes it is simple: fixing it by hand costs more than living with it wrong. A manual inventory of two hundred devices goes stale faster than one person can walk through it. This is not a discipline problem in the network team. The method does not scale, and no amount of human effort fixes that.

An inventory that needs human effort to stay current was already out of date the day it was written.

An inventory is not a source of truth

The terms get used interchangeably, and they are not the same. The difference matters because it determines what you can build on top.

An **inventory** answers the question *what is there?*. It is a list: devices, models, versions, locations. It is useful and it is the first step, but it is descriptive. It tells you what whoever compiled it happened to find.

A **source of truth** answers a different question: *what should be there, and in what state?*. It is the declared reference against which reality is compared. It does not describe the network; it defines how the network is supposed to be. That difference is what turns a document into an operational tool.

With an inventory you can say "we have forty switches in Mérida". With a source of truth you can say "we have forty switches in Mérida, thirty-eight meet the organisation's standard, and the two that do not fail on

this specific configuration line". The second answer is the one that holds up in an audit, in a tender, and in a budget meeting.

The gap between the two is not a matter of format. A source of truth requires that an intent has been declared: which services each device should have based on its site, role, platform, or model. Without that declaration there is nothing to compare against, and without comparison there is no compliance, only description.

Multi-vendor is where most tools break

This is where most tools start telling you things that are not quite true, and it is worth understanding why.

Every vendor models configuration differently, and not superficially. Cisco IOS expresses management access control as a standard list applied to the VTY lines. Junos protects its control plane with a firewall filter on the loopback interface, where term order decides the outcome because first match wins. FortiGate solves it with trusted hosts on the administrator profile. All three do the same thing conceptually. None of them does it the same way.

The usual shortcut is to translate everything into one common syntax, almost always Cisco's, because that is what most people know. It works well enough to render a tidy table, and it fails at the moment that matters: when configuration has to be *generated* or *compared* against what is actually running. The translation discards exactly what makes each platform different, and what gets discarded is usually what decides whether a command applies or is rejected.

A genuinely multi-vendor source of truth has a model per vendor, not a translation into a common dialect.

The cost of doing it properly is real: it means maintaining the model, the templates, and the compliance rules for each platform separately. The benefit is that what the tool asserts about a Junos device is true in Junos terms, and not a convenient approximation.

What becomes possible when the network describes itself

A living source of truth — fed by discovery, not by manual entry — stops being a document and becomes the foundation for things that cannot be built without it:

- **Configuration compliance.** Comparing what is there against what was declared, by site, role, or device, with the exact line that backs every finding.
- **Topology and path tracing.** The real path between two addresses comes from collected routing tables, not from a hand-drawn diagram that has aged.
- **Addressing intelligence.** Which subnets exist, which overlap, and which access port an address lives on, by cross-referencing the ARP table with the MAC table.
- **Answering audits.** Evidence stops being assembled by hand the week before and becomes a query.
- **Integration with the rest of the ecosystem.** This is the one most often underestimated, and it deserves its own paragraph.

Security tooling, address management platforms such as Infoblox, ticketing systems, and executive dashboards all have something in common: **they all need to know what is on the network, and none of them finds out on its own**. Each maintains its own partial copy, fed by its own method, and no two copies agree.

Once a source of truth exists that can be queried through an API, that problem changes shape. Instead of five inventories disagreeing, there is one that the others consume. The security team stops asking the network team which devices sit in a segment and queries it instead. And the argument over which of the two lists is correct simply goes away.

But inventory alone closes nothing

This is worth saying plainly, because it is where many projects stop, satisfied: having the source of truth is the beginning, not the end.

A network can be perfectly inventoried, with its configuration meeting the standard one hundred per cent, and still be broken. A routing protocol's configuration can be exactly what was declared while three of the interfaces participating in it are down. Configuration compliance does not see it, because the configuration is fine. What is wrong is the state.

That is the second leap, and the subject of the next article: the difference between configuration compliance and state compliance, why correlating them changes what you can credibly claim about your network, and why almost no tool does it.

How Oktavia handles it

Oktavia builds the source of truth through discovery across Cisco IOS and IOS-XE, Fortinet FortiGate and FortiSwitch, and Juniper Junos, with a normalised configuration model *per vendor* rather than a translation into a common syntax. Inventory, topology, addressing, and intended configuration live in the same base, queried from the interface or through an API.

Intent is declared by scope — defaults, by site type, by site, by platform, by model, by role, and by device — so a standard is expressed once and applies to everything it covers, instead of being repeated device by device.

Want to see what your network looks like?

To quote, all we need is roughly how many routers, switches, firewalls, and controllers you manage. No detailed inventory required — if you had one, you probably would not be reading this.

About Oktanet

Oktanet is a Mexican network automation company focused on controlled, traceable, and scalable operations for enterprise environments across Latin America.

[LinkedIn](#) · [YouTube](#)

© 2026 Oktanet · Oktavia · Public article, with no confidential customer information.